Project Design Modelling

Hostel Management System

Team **RBMA**

Member Name	Student ID
Rozh Khals	01-24-00463
Ari Abdulmajeed	01-23-00758
Bawan Mustafa	
Muhammad Hussein Hassan	01-23-00769
Muhammad Bayar	

Project Foundation & User Stories

The following are the basic components taken from our P2 document that are used for the models created in this assignment:

Key System Feature

Check-in and Bed Assignment (Modelled in Tasks 2 and 4b)

Life Cycle Object

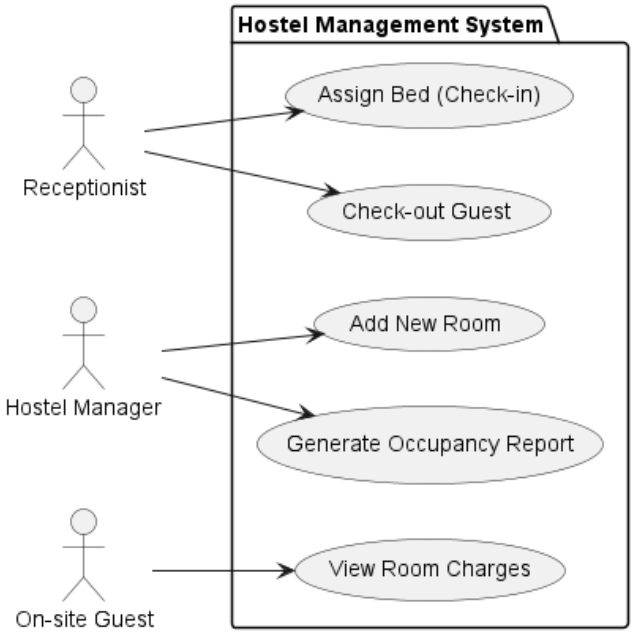
Booking (Modelled in Task 3)

Selected User Stories

- US-01: As a receptionist, I want to view all daily check-ins so that I can process bed assignments quickly.
- US-02: As a hostel manager, I want to add new rooms to the system so that the assignable inventory is accurate.
- US-03: As an on-site guest, I want to view my current room charges so that I can track my expenses.
- US-05: As a hostel manager, I want to generate a real-time daily occupancy report.
- US-06: As a receptionist, I want to check out a guest to free up their bed.

Task 1: Use Case Model

Using this diagram, we find the scope of the Hostel Management System along with its interaction partners, Receptionist, Hostel Manager, and the On-site Guest.



Key Use Case Description

Field	Description
Actors	Receptionist
Description	The receptionist selects a pending daily check-in and assigns an available bed to the arriving guest.
Data	Booking ID, Guest Name, Bed Number, Timestamp
Stimulus	A guest arrives at the hostel reception desk requesting to check in.
Response	The system changes the guest's status to "active", marks the selected bed as "occupied" in the inventory, and displays a success confirmation.
Comments	The interface must allow the receptionist to complete this entire process in under two minutes (Performance NFR).

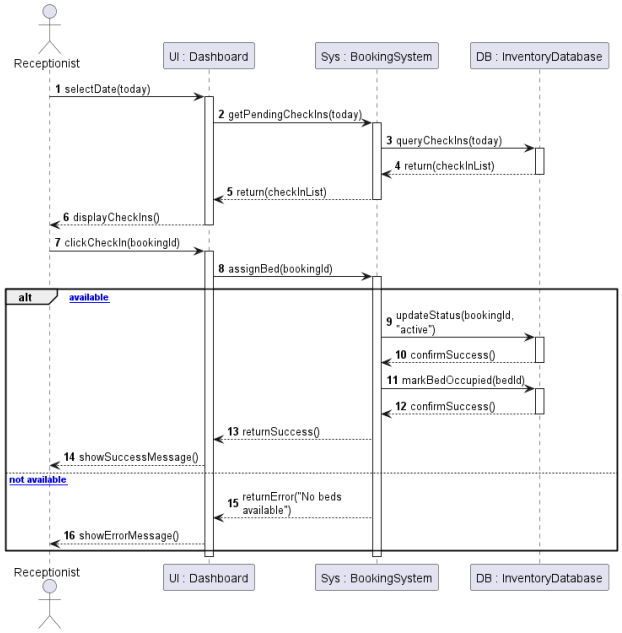
User Story Traceability

- Assign Bed (Check-in): Covers US-01
- Add New Room: Covers US-02
- View Room Charges: Covers US-03

- Generate Occupancy Report: Covers US-05
- Check-out Guest: Covers US-06

Task 2: Sequence Diagram

This sequence diagram showss the interaction needed to implement the important feature: Guest Check-in and Bed Assignment.



Reading the Diagram (Execution Flow)

1. The Receptionist initiates the process by calling selectDate(today) on the Dashboard UI.

2. The Dashboard UI requests the pending check-ins from the Booking System via getPendingCheckIns(today).

3. The Booking System queries the Inventory Database using queryCheckIns(today).

4. The Database returns the checkInList, which propagates back up through the System and UI to be displayed.

5. The Receptionist clicks on a specific guest to check them in, triggering clickCheckIn(bookingId).

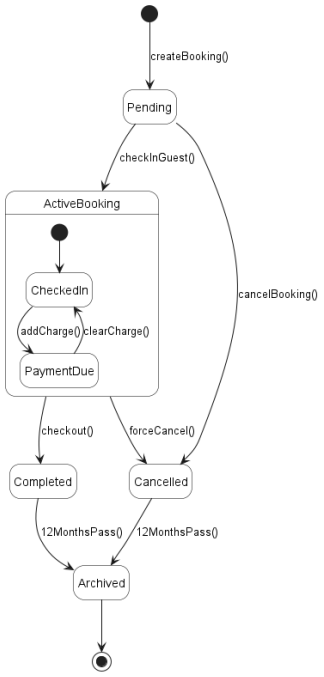
6. The Dashboard UI passes the assignBed(bookingId) command to the Booking System to process the logic.

7. [Alt - bed available]: If a bed is available, the Booking System updates the booking status to active and marks the assigned bed as occupied in the Database. A success message is cascaded back to the Receptionist.

8. [Alt - bed not available]: If the system detects a conflict and no beds are available, the Booking System returns a "No beds available" error, which the UI displays to the Receptionist.

Task 3: State Machine Diagram

This figure describes the life cycle of the Booking object from its creation up to archiving.



States and Stimuli

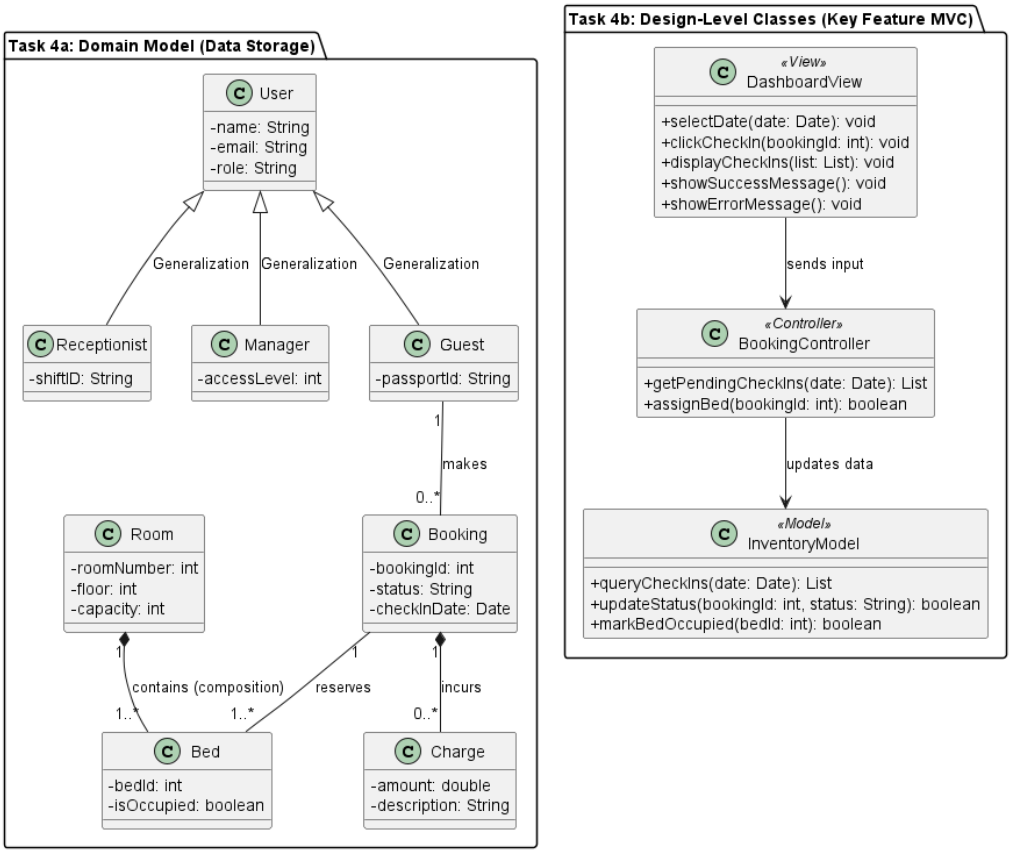
State	Description	Stimuli (Exit Events)
Pending	The booking is created, but the guest has not arrived yet.	checkInGuest(), cancelBooking()
ActiveBooking (Superstate)	The guest is actively occupying a bed at the hostel.	checkout(), forceCancel()
CheckedIn (Substate)	The guest is present, in their assigned bed, and has a zero balance.	addCharge() (moves to PaymentDue), or inherits superstate events.
Payment Due (Substate)	The guest has accumulated unpaid room charges.	clearCharge() (moves to CheckedIn), or inherits superstate events.
Completed	The guest has successfully checked out and settled all bills.	12MonthsPass()
Cancelled	The booking was terminated (either a no-show or forced eviction).	12MonthsPass()
Archived	The booking data is	(Automatically transitions

	scrubbed and archived to comply with privacy laws.	to Final State)

Task 4: Class Diagrams

Static structure of the system is shown below.

Task 4a shows the persistent data store (Domain Model) whereas Task 4b explains the application logic that supports the critical check-in functionality (Design-Level MVC).



Task 5: Consistency Check and Justification

1. Consistency Matrix

Check	Where in your models	Result
Every sequence message arriving at an object is an operation of its class	Task 2 & Task 4b	OK (e.g., assignBed aligns)
Every event in the state diagram is a message or user action elsewhere	Task 3 & Task 2	OK (e.g., checkInGuest aligns)
Every use case traces to at least one user story	Task 1 & P2 Stories	OK (Traced in Task 1)
Every class in the domain model appears in at least one sequence diagram or use case	Task 4a & Tasks 1, 2	OK (Booking, Bed, Guest traced)
Actors in the use case diagram have corresponding roles/classes in the domain model	Task 1 & Task 4a	OK (Receptionist, Manager, Guest)
The selected key feature (Check-in) is represented accurately in the sequence diagram	Project Scope & Task 2	OK
States in the state machine align with the 'status' attribute in the Booking class	Task 3 & Task 4a	OK (ActiveBooking maps to status)
UI inputs in the sequence diagram match operations in the View class	Task 2 & Task 4b	OK (selectDate matches View)

2. Model Justification

In the case of this project, we consciously decided to limit the Domain Model to 8 classes and left out other models such as Activity Diagrams, Communication Diagrams, and Deployment Diagrams. According to Sommerville, the lesser number of models used, the lesser the overall cost and time of design will be. Since our group follows the Agile/Scrum methodology (as explained in our P3 Planning document), we value the working software over documentation. The Sequence Diagram and Design-Level Class Diagram (Tasks 2 and 4b) were the most important diagrams to elaborate further since they relate directly to our MVC architecture and give a clear understanding of the exact Java methods we need for our backend. Focusing only on the essential models and limiting ourselves to the core check-in feature allows us not to over-engineer the system.

Team Contribution Table

Member Name	Contributions	Percentage
Rozh	Task 1 (Use Case), Task 5 (Consistency Check), Final Review	30%
Ari	Task 2 (Sequence Diagram), Technical Writing	20%
Muhammad Hussein	Task 3 (State Machine Diagram), Markdown Formatting	20%
Bawan	Task 4 (Class Diagrams), Domain Model Logic	15%
Muhammad Bayar	Task 5 (Consistency Check & Reflection)	15%